Bilibili网站“每周必看”栏目数据分析

https://dblab.xmu.edu.cn/blog/2738/

一、实验目的

Bilibili(简称b站) 是中国年轻人高度聚集的文化社区和视频平台，人们可以通过弹幕、评论、点赞或收藏等方式来与视频发布者(称为up主)来进行互动。《每周必看》是b站的一个栏目， b站会对新一周发布的视频根据不同条件进行筛选，将热门的、有趣的、有价值的视频收录在该栏目中推荐给用户观看。这次实验中，希望对被收录在b站《每周必看》视频与up主的数据信息进行研究，通过运用大数据处理框架 Spark、Hadoop 及数据可视化技术，对这些数据进行存储、处理和分析，并对每个收录视频能否进入热搜榜前10进行分类。具体来说本次作业实现功能如下：

(1)采用爬虫技术，编写代码收集截止至5月20日b站所有《每周必看》栏目的数据；

(2)对原始数据进行清洗，包括有价值字段的选择、异常数据的删除以及文本数据的修正等，最后数据保存为txt文件并上传至HDFS；

(3)使用spark sql组件对HDFS的数据进行分析，主要统计各收录视频的基本播放情况以及up主的累计数据；

(4)使用spark MLlib组件对HDFS的数据进行分析，研究视频点赞数、播放量、互动热度等之间的关系，并训练机器学习模型对视频能否进入热搜榜前十进行分类；

(5)使用pyecharts工具对分析结果进行可视化。

二、实验环境

本次实验在Linux虚拟机中用Pycharm平台python语言实现，主要环境如下，点击链接可跳转至相应安装教程，其他需要的第三方库及对应版本可见requirement.txt：
(1)Linux: Ubuntu 22.04.2 LTS amd64
(2)Python: 3.8.16
(3)Hadoop: 3.1.3
(4)Spark: 3.2.0
(5)Anaconda: 4.10.1

三、操作流程

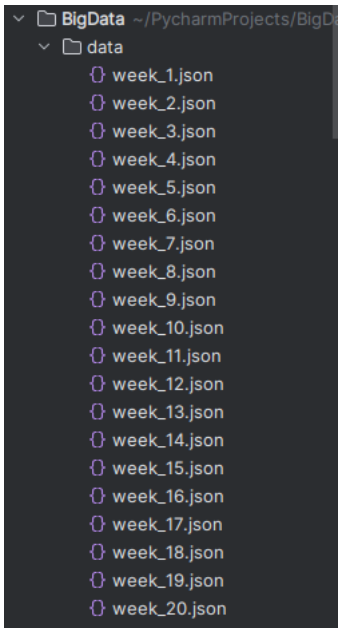
1、数据集收集

爬虫代码在bilibili_week.py文件中，可在Pycharm中直接执行。b站提供了API接口以便开发者获得每一期《每周必看》的视频数据(https://api.bilibili.com/x/web-interface/popular/series/one?number={},花括号内填入要收集数据的期数)，直接使用requests库发送请求并将得到的响应进行保存即可，在发送请求时还添加了headers将其伪装为浏览器访问。
但是重复使用相同的user-agent很容易被网站识别为爬虫程序，借助python中fake_useragent第三方模块，在每一次发送请求时随机使用一个UA，并且设置retry让每一次的代码等到成功运行当前数据的爬虫后才能进行下一步操作。爬取后的数据直接转为json格式并保存下来，最终的爬虫主要代码如下：

```python
@retry( )
def getWeek_json(url,json_path):
    # 随机User-Agent
    random_UA = UserAgent().random
    # headers信息，添加user-agent和cookies
    headers = {
        "User-Agent": random_UA,
        "referer": "https://www.bilibili.com/",
        "origin": "https://www.bilibili.com/"
    }
    # 获取响应，转为json格式并保存
    response = requests.get(url=url,headers=headers,timeout=10)
    response_data = response.json()
    with open(json_path, 'w',encoding='utf-8') as f:
        json.dump(response_data, f,ensure_ascii=False)
    # 休眠，确保不会被反爬
    time.sleep(1)
```

B站截止至2023年5月19日一共有217期《每周必看》，因此设置共爬取217期，爬取的所有json文件都保存至当前目录的./data目录下。

```python
if __name__=='__main__':
    # 官方api
    url='https://api.bilibili.com/x/web-interface/popular/series/one?number={}'
    # 爬虫数据存储路径
    data_folder = './data'
    os.makedirs(data_folder, exist_ok=True)
    # 开始爬虫
    for i in range(1,218):
        URL = url.format(str(i))
        # 每周数据存储路径
        json_fpath = os.path.join(data_folder,'week_{}.json'.format(str(i)))
        getWeek_json(URL,json_fpath)
```

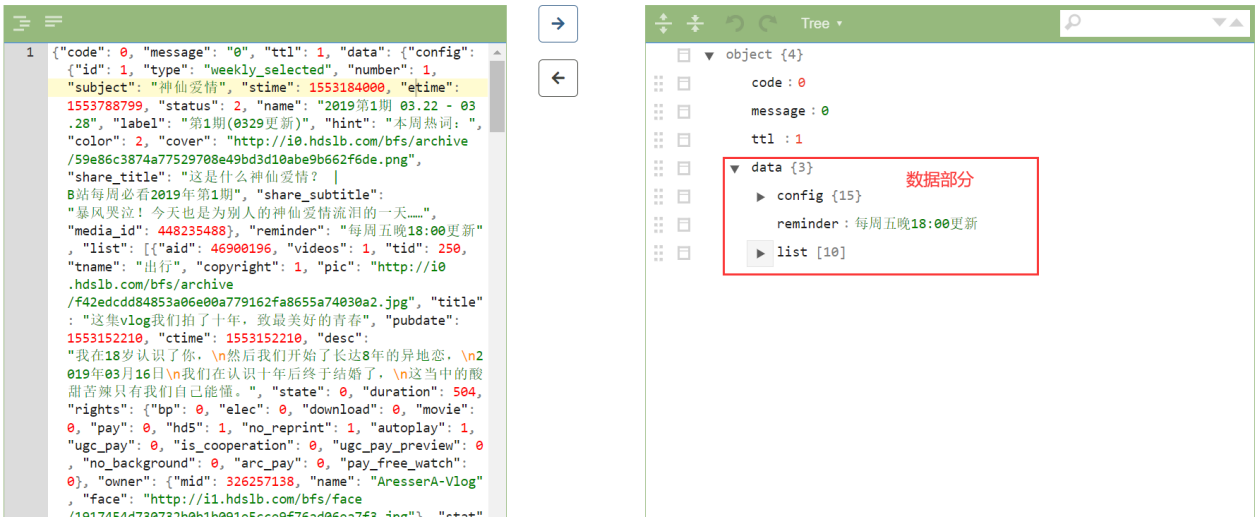


可以点击这里从百度网盘下载爬取到的B站数据集和本案例的所有代码。

2、数据预处理

数据预处理阶段包括了有效数据的选择、异常数据与空白数据的处理、文本数据的处理，这一步的代码在data_preprocess.py中，可在Pycharm中直接执行。。

(1)数据选择：打开上一步爬虫得到的其中一个json数据文件，借助菜鸟工具提供的json格式化工具，查看爬取到的数据格式，返回的信息包含在data中：



data包括3个部分的信息，其中config包括这一期视频栏目的整体信息，包括期数、时间、封面视屏内容等，其中config.name记录每一个视频的所属期数。

config {15}

id : 1

type : weekly_selected

number : 1

subject : 神仙爱情

stime : 1553184000

etime : 1553788799

status : 2

name : 2019第1期 03.22 - 03.28

label : 第1期(0329更新)

hint : 本周热词:

color : 2

cover : http://i0.hdslb.com/bfs/archive/59e86c3874a77529708e49bd3d10abe9b662f6de.png

share_title : 这是什么神仙爱情？ | B站每周必看2019年第1期

share_subtitle : 暴风哭泣！今天也是为别人的神仙爱情流泪的一天……

media_id : 448235488

reminder : 每周五晚18:00更新

list则是这一期栏目每一个收录视频的具体情况，包括视频的标题、描述以及推荐理由：



发布者的信息则包含在元素的owner部分：



视频的观看次数、弹幕数、转发数等则包含在stat中：



除了这些信息，其余数据在本项目中无意义，因此对单个json文件，选取上述几个字段，将有用的信息保存在dataframe：

```
key = ['up', 'time', 'title', 'desc', 'view', 'danmaku', 'reply', 'favorite',\
       'coin', 'share', 'like', 'dislike','rcmd_reason', 'tname', 'his_rank']
_df = pd.DataFrame(columns=key)
_week = pd.read_json(json_fpath)

# 获取每个必看视频基础的信息
_base_info = pd.DataFrame(_week['data']['list'])
_df[['tname', 'title', 'desc', 'rcmd_reason']] = _base_info[['tname', 'title', 'desc', 'rcmd_reason']]
# 获取up主的信息
_owner = _base_info['owner'].values.tolist()
_owner = pd.DataFrame(_owner)
_df['up'] = _owner['name']
# 获取视频的播放信息
_stat = _base_info['stat'].values.tolist()
_stat = pd.DataFrame(_stat)
_df[['view', 'danmaku', 'reply', 'favorite', 'coin', 'share', 'his_rank', 'like', 'dislike']] = \
    _stat[['view', 'danmaku', 'reply', 'favorite', 'coin', 'share', 'his_rank', 'like', 'dislike']]
_df['time'] = _week['data']['config']['name']
```

(2)异常数据的处理：

首先删除包含空值和重复的数据，其中当up主名字和视频标题相同视为重复；认为观看数view、弹幕数danmaku、评论数reply、收藏数favorite、投币数coin、分享次数share、点赞人数like、历史排名his_rank小于或等于0，不喜欢的人数dislike超过0的数据为异常数据，一并删除。上述数据删除之后，dislike字段也无意义，同样删除。这一步骤的代码如下：

```
# 删除掉包含有空值的数据行
dataframe.dropna(how='any', axis=0, inplace=True)
# 删除掉重复的数据，其中当视频标题与作者名称相同认为数据重复
dataframe = dataframe.drop_duplicates(subset=['title', 'up'])
# 删除掉异常的数据，认为观看数、弹幕数、回复、喜欢、投币、分享、收藏数小于或等于0，不喜欢的人数超过0则认为数据异常
incorrect_df = dataframe.loc[(dataframe['view'] <= 0) \
                             | (dataframe['danmaku'] <= 0) \
                             | (dataframe['reply'] <= 0) \
                             | (dataframe['favorite'] <= 0) \
                             | (dataframe['coin'] <= 0) \
                             | (dataframe['share'] <= 0) \
                             | (dataframe['like'] <= 0) \
                             | (dataframe['dislike'] > 0) \
                             | (dataframe['his_rank'] <= 0)]
# 丢弃异常数据
dataframe = dataframe.drop(incorrect_df.index)
# dislike数据无意义，丢弃
dataframe = dataframe.drop(columns=['dislike'])
```

(3)文本数据的处理：

入选栏目的某些视频描述desc和推荐理由rcmd_reason为空，使用视频的标题对其进行填充，并且为了最后数据保存的规范性，对一些特殊符号例如换行符、Tab等进行处理：

```
# 处理描述为空的数据，使用该视频的标题进行填充
desc_none = dataframe.loc[(dataframe['desc'] == '') | (dataframe['desc'] == '-')]
for i in range(desc_none.shape[0]):
    dataframe.loc[desc_none.index[i], 'desc'] = dataframe.loc[desc_none.index[i], 'title']
# 处理推荐理由为空的数据，使用该视频的标题进行填充
rcmd_reason_none = dataframe.loc[(dataframe['rcmd_reason'] == '') | (dataframe['rcmd_reason'] == '-')]
for i in range(rcmd_reason_none.shape[0]):
    dataframe.loc[rcmd_reason_none.index[i], 'rcmd_reason'] = dataframe.loc[rcmd_reason_none.index[i], 'title']
```

```
# 对长文本进行处理
dataframe['title'] = dataframe['title'].map(lambda x: x.replace(",", " "))
dataframe['desc'] = dataframe['desc'].map(lambda x: x.replace(",", " "))
dataframe['rcmd_reason'] = dataframe['rcmd_reason'].map(lambda x: x.replace(",", " "))
dataframe['title'] = dataframe['title'].map(lambda x: x.replace(";", " "))
dataframe['desc'] = dataframe['desc'].map(lambda x: x.replace(";", " "))
dataframe['rcmd_reason'] = dataframe['rcmd_reason'].map(lambda x: x.replace(";", " "))
dataframe['title'] = dataframe['title'].map(lambda x: x.replace("\n"," "))
dataframe['desc'] = dataframe['desc'].map(lambda x: x.replace("\n"," "))
dataframe['rcmd_reason'] = dataframe['rcmd_reason'].map(lambda x: x.replace("\n"," "))
dataframe['title'] = dataframe['title'].map(lambda x: x.replace("\r"," "))
dataframe['desc'] = dataframe['desc'].map(lambda x: x.replace("\r"," "))
dataframe['rcmd_reason'] = dataframe['rcmd_reason'].map(lambda x: x.replace("\r"," "))
dataframe['title'] = dataframe['title'].map(lambda x: x.replace("\t"," "))
dataframe['desc'] = dataframe['desc'].map(lambda x: x.replace("\t"," "))
dataframe['rcmd_reason'] = dataframe['rcmd_reason'].map(lambda x: x.replace("\t"," "))
```

(4)数据合并与上传

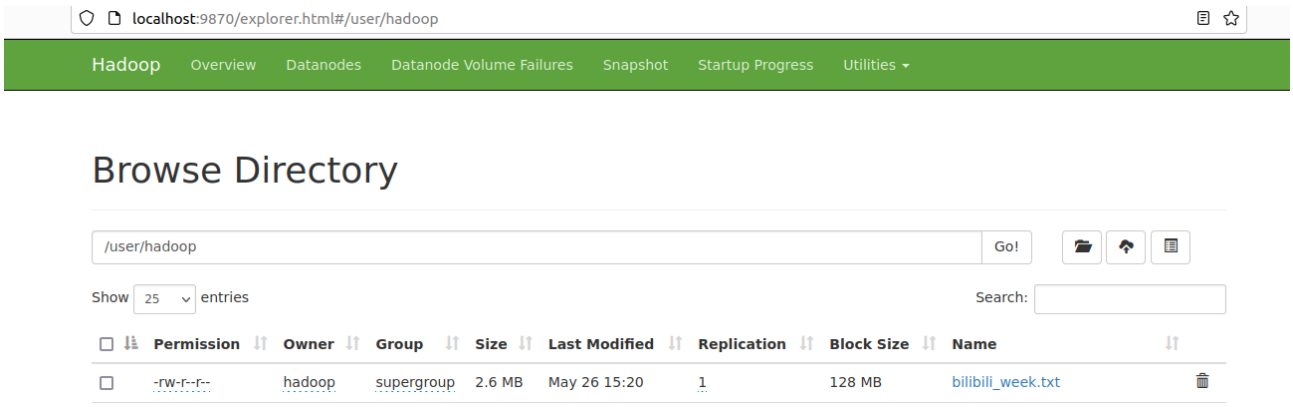
对每一个文件重复进行上述操作，并不断合并得到的dataframe，最后将数据去除表头，保存为文本文件，命名bilibili_week.txt：

```python
def merge_data(json_data, save_path):
    key = ['up', 'time', 'title', 'desc', 'view', 'danmaku', 'reply', 'favorite', \
           'coin', 'share', 'like', 'dislike','rcmd_reason', 'tname', 'his_rank']
    df = pd.DataFrame(columns=key)
    # 依次处理每周的数据并进行合并
    for each_json in json_data:
        _df = select_data(each_json)
        df = pd.concat([df, _df], ignore_index=True)
    # 进行数据处理
    df = clean_data(df)
    # 保存为txt文件
    df.to_csv(save_path, header=None, index=None, sep='\t', mode='w')

if __name__ == '__main__':
    # 读取所有的json数据
    json_data = glob(os.path.join('./data', '*.json'))
    #最后的数据保存路径
    save_dir = 'bilibili_week.txt'
    # 进行数据合并并存储为txt文件
    merge_data(json_data, save_dir)
```

在Linux 终端输入./bin/hdfs dfs -put /home/Hadoop/PycharmProjects/BigData/bilibili_week.txt /user/hadoop，将该文件上传至HDFS，通过ls命令和网页浏览，可以验证数据成功上传：

```
(base) hadoop@hadoop-virtual-machine:/usr/local/hadoop$ ./bin/hdfs dfs -put /hom
e/hadoop/PycharmProjects/BigData/bilibili_week.txt /user/hadoop
2023-05-26 15:19:58,862 INFO sasl.SaslDataTransferClient: SASL encryption trust
check: localHostTrusted = false, remoteHostTrusted = false
(base) hadoop@hadoop-virtual-machine:/usr/local/hadoop$ ./bin/hdfs dfs -ls
Found 2 items
-rw-r--r--   1 hadoop supergroup    2721738 2023-05-26 15:20 bilibili_week.txt
-rw-r--r--   1 hadoop supergroup       1051 2023-05-25 23:28 goodreads_data_proc
essed.csv
(base) hadoop@hadoop-virtual-machine:/usr/local/hadoop$
```



至此，数据清洗完成。

3、数据分析

进行数据分析前，需要将HDFS上的数据加载成RDD，然后再由RDD转为Dataframe。这一步的实现步骤为：

①创建SparkSession和SparkContext对象，并用textFile函数读取HDFS上的文本数据，并将其存储为RDD；

②用map函数对RDD的每个元素进行切割并转化为包含多个字段的Row对象；

③定义数据集的结构schema，为Dataframe建立表头，将RDD转化为一个Dataframe。

④在使用sql组件的时候还需要将DataFrame注册为Spark SQL中的临时视图。

以上操作对应的代码如下：

```
def initialize(txt_file):
    # 创建SparkContext和SparkSession对象
    sc = SparkContext('local', 'spark_project')
    sc.setLogLevel('WARN')
    spark = SparkSession.builder.getOrCreate()
    # 把数据加载为RDD，并且对每一行数据进行切割，转为Row对象
    rdd = spark.sparkContext.textFile(txt_file)\
        .map(lambda x: x.split("\t")).map(lambda x:Row(x[0],x[1],x[2],x[3],int(x[4]),int(x[5]),int(x[6]),\
                                                       int(x[7]),int(x[8]),int(x[9]),int(x[10]),x[11],x[12],int(x[13])))
    # 定义数据集结构
    fields = [StructField("up", StringType(), False), StructField("time", StringType(), False),
              StructField("title", StringType(), False),StructField("desc", StringType(), False),
              StructField("view", IntegerType(), False), StructField("danmaku", IntegerType(), False),
              StructField("reply", IntegerType(), False), StructField("favorite", IntegerType(), False),
              StructField("coin", IntegerType(), False), StructField("share", IntegerType(), False),
              StructField("like", IntegerType(), False), StructField("rcmd_reason", StringType(), False),
              StructField("tname", StringType(), False), StructField("his_rank", IntegerType(), False), ]
    schema = StructType(fields)
    # 将RDD转为Dataframe
    data = spark.createDataFrame(rdd, schema)
    # 注册sql临时视图
    data.createOrReplaceTempView("data")
    return spark
```

(1)基于spark sql组件的分析

这一部分的代码都在data_analysize1.py文件中，通过spark-submit data_analysize1.py 执行代码。

①统计视频收录次数最多的up主

b站up主的名字不可以重复，因此每个up主由字段'up'唯一标识，数据中up主视频被收录的总次数为count(up)，要求得到的结果根据名字up分组并按照入选的次数降序排序，筛选出10个视频入选次数最多的up，得到的popular_up为spark.DataFrame类型，再使用

toPandas将其转为pandas.Dataframe并通过to_csv函数把结果保存在csv文件中。这一过程的代码如下：

```
def top_popular_up(spark,base_dir):
    popular_up = spark.sql(
        "SELECT up,COUNT(up) AS popular_up_times FROM data GROUP BY up ORDER BY popular_up_times DESC LIMIT 10")
    data = popular_up.toPandas()
    save_dir = os.path.join(base_dir,'top_popular_up.csv')
    data.to_csv(save_dir,index=False)
```

②统计入选次数最多的视频分类
与①类似，tname表示入选视频的视频分区，count(tname)表示属于该种分区视频的收录次数，要求得到的结果根据tname分组并按照入选的次数降序排序，筛选出10个收录次数最多的视频分区，得到的popular_subject用toPandas将其转为pandas.Dataframe并通过to_csv函数把结果保存在csv文件中。这一过程代码如下：

```
def top_popular_subject(spark,base_dir):
    popular_subject = spark.sql(
        "SELECT tname,COUNT(tname) AS popular_subject_times FROM data GROUP BY tname ORDER BY popular_subject_times DESC LIMIT 10")
    data = popular_subject.toPandas()
    save_dir = os.path.join(base_dir,'top_popular_subject.csv')
    data.to_csv(save_dir, index=False)
```

③播放量数据统计
对于视频播放量的统计，选取视频标题title和播放量view的字段，对查询结果按照view字段进行降序排序并筛选播放量最多的10个视频，将其转为Pandas.dataframe并保存在csv文件中，对应代码如下：

```
view_data = spark.sql(
    "SELECT title,view AS view_data FROM data ORDER BY view DESC LIMIT 10")
data = view_data.toPandas()
save_dir = os.path.join(base_dir, 'video_view_data.csv')
data.to_csv(save_dir,index=False)
```

对于up主所有收录视频的累计播放量统计，选取up主名字up并计算每个up主的总播放量sum(view)，对查询结果按照up的名称进行分组，并按照总播放量sum(view)降序排序并筛选总次数最多的10个up主名字，查询结果同样通过toPandas()和to_csv()保存在csv文件中，代码如下：

```
view_data = spark.sql(
    "SELECT up,sum(view) AS view_data FROM data GROUP BY up ORDER BY sum(view) DESC LIMIT 10")
data = view_data.toPandas()
save_dir = os.path.join(base_dir, 'up_view_data.csv')
data.to_csv(save_dir,index=False)
```

弹幕数、视频回复量、收藏次数、投币数、分享次数和点赞次数最多的前10个视频和up主的数据统计与上述操作类似，只需要修改sql语句中select相应的字段，这里就不再展开。

④词频统计

为了研究标题、视频简介、推荐理由等常用词，需要进行词频统计，以视频标题分析为例，对视频标题的词频分析步骤如下：

```
wordCount_title = spark.sql("SELECT title from data").rdd.flatMap(
    lambda line: pretty_cut(line['title'])).map(lambda word: (word, 1)).reduceByKey(
    lambda a, b: a + b).repartition(1).sortBy(lambda x: x[1], False)
```

上面的代码首先通过sql语句查询data视图中的title字段，返回一个DataFrame对象，然后通过rdd.flatMap(lambda line: pretty_cut(line['title']))将DataFrame转换为RDD，并且RDD的title字段通过pretty_cut进行分词，并将结果展平为一个新的RDD。其中pretty_cut的代码如下，该函数使用jieba对每个中文语句进行分词，并去除停用词：

```
def pretty_cut(sentence):
    cut_list = jieba.lcut(''.join(re.findall('[\u4e00-\u9fa5]', sentence)), cut_all=False)
    for i in range(len(cut_list) - 1, -1, -1):
        if cut_list[i] in stopwords:
            del cut_list[i]
    return cut_list
```

每一行数据分词之后，再通过map(lambda word: (word, 1)) 将每个词映射为一个键值对，key是单词，1表示单词出现的次数是1，reduceByKey(lambda a, b: a + b)则对RDD中的每个键值对进行聚合，将相同键的值相加，得到每个单词出现的总次数。repartition将RDD的分区数设置为1， sortBy会按照每个单词出现的次数进行降序排序，得到出现频率最高的单词。

简单起见，在将RDD转为Dataframe后，只选取词频前300且不为空的词语，最后保存为csv文件：

```
# 转为Dataframe
wordCountSchema = StructType([StructField("word", StringType(), True), StructField("count", IntegerType(), True)])
wordCountDF = spark.createDataFrame(wordCount_title, wordCountSchema)
# 过滤掉空值
wordCountDF = wordCountDF.filter(wordCountDF["word"] != '')
# 只保留频数前300的单词
wordCount_title_300 = wordCountDF.take(300)
wordCount_title_300 = spark.createDataFrame(wordCount_title_300, wordCountSchema).toPandas()
# 保存为csv文件
save_dir = os.path.join(base_dir, 'title_word.csv')
wordCount_title_300.to_csv(save_dir, index=False)
```

视频简介、入选推荐理由的文本数据分析操作同上，这里也不再展开。
最后main函数依次执行上述的函数，得到的结果保存在static文件中：

```
if __name__ == '__main__':
    # HDFS文件路径
    txt_file = 'hdfs://localhost:9000/user/hadoop/bilibili_week.txt'
    # 初始化
    spark = initialize(txt_file)
    # 分析结果的存储路径
    base_dir = 'static/'
    if not os.path.exists(base_dir):
        os.makedirs(base_dir,exist_ok=True)
    # 开始进行分析
    # 分析视频入选最多的up主
    top_popular_up(spark, base_dir)
    # 分析视频入选最多的视频分区
    top_popular_subject(spark, base_dir)
    # 分析播放量
    top_popular_view(spark,base_dir)
    # 分析弹幕数
    top_popular_danmaku(spark, base_dir)
    # 分析评论数
    top_popular_reply(spark, base_dir)
    # 分析收藏数
    top_popular_favorite(spark, base_dir)
    # 分析投币数
    top_popular_coin(spark, base_dir)
    # 分析分享次数
    top_popular_share(spark, base_dir)
    # 分析点赞数量
    top_popular_like(spark, base_dir)
    # 分析词频
    word_count(spark, base_dir)
```

(2)基于spark MLlib组件的分析
这一部分的代码都在data_analysize2.py文件中，通过spark-submit data_analysize2.py 执行代码。
①数据处理
读取到HDFS数据之后，选择其中几组字段：

['view','danmaku','reply','favorite','coin','share','like']进行分析，研究这几组变量与当周历史排名his_rank之间的关系。通常情况下人们比较关心热搜榜前10的数据，因此将数据划分为两类并新增label标签：1说明该条视频曾经进入b站热搜榜前10，0说明该视频不曾进入热搜榜前十。数据筛选和新增对应代码如下：

```
# 删除掉无用的数据
df = df.drop('up')
df = df.drop('time')
df = df.drop('title')
df = df.drop('desc')
df = df.drop('rcmd_reason')
df = df.drop('tname')
# 根据历史排名his_rank，新增类别标签label
df = df.withColumn('label', when(df.his_rank <= 10, 1).otherwise(0))
```

为了方便后续使用机器学习模型进行训练和验证，使用VectorAssembler类将数据转化为特征向量：

```
# 将数据转为特征向量
required_features = ['view', 'danmaku', 'reply', 'favorite', 'coin', 'share', 'like']
assembler = VectorAssembler(
    inputCols=required_features,
    outputCol='features')
transformed_data = assembler.transform(df)
```

最后对数据按照8:2划分训练集和验证集：

```
#对数据进行划分
(training_data, test_data) = transformed_data.randomSplit([0.8, 0.2], seed=2023)
print("训练数据集总数: " + str(training_data.count()))
print("测试数据集总数: " + str(test_data.count()))
return transformed_data,training_data,test_data
```

此时训练集与验证集的样本数为：

```
训练数据集总数: 5463
测试数据集总数: 1367
```

②变量的相关性

使用pyspark.ml.stat中的Correlation，计算特征之间的斯皮尔曼系数，并将结果保存在csv文件中：

```
def corr_matrix(df,cor_save_dir):
    cor_mat = Correlation.corr(df, "features", "spearman").head()[0]
    cor_df = pd.DataFrame(cor_mat.toArray())
    cor_df.columns = ['view','danmaku','reply','favorite','coin','share','like']
    cor_df.to_csv(cor_save_dir, index=False)
```

③分类器的训练

根据['view','danmaku','reply','favorite','coin','share','like']和label标签，训练分类器，让分类器根据视频的播放量、评论数等判断它是否可以进入热搜榜前10。pyspark.ml.classification提供多种封装好的分类模型，在这次实验中选择逻辑回归、决策树、随机森林和GBT进行实验。

以逻辑回归分类器为例，首先定义分类器：设置标签列为label，特征列为features，并且逻辑回归最大迭代15次。

```
# 实例化逻辑回归算法
lr = LogisticRegression(labelCol='label',featuresCol='features',maxIter=15)
```

使用训练集进行训练后，对验证集进行测试：

```
# 进行模型训练
model = lr.fit(training_data)
# 进行模型验证
lr_predictions = model.transform(test_data)
```

为了研究分类器的性能，使用MulticlassClassificationEvaluator统计模型在验证集上的分类正确率，使用BinaryClassificationEvaluator得到模型二分类的auc。这两个数值越高，说明模型性能越好。

```
# 计算分类acc
multi_evaluator = MulticlassClassificationEvaluator(
    labelCol='label', metricName='accuracy')
acc = multi_evaluator.evaluate(lr_predictions)
print('LogisticRegression classifier Accuracy:{:.4f}'.format(acc))
# 计算模型auc
binary_evaluator = BinaryClassificationEvaluator(rawPredictionCol="rawPrediction", labelCol="label",
    metricName="areaUnderROC")
auc = binary_evaluator.evaluate(lr_predictions)
print('LogisticRegression classifier Auc:{:.4f}'.format(auc))
```

其余三种分类器(决策树算法、随机森林算法和GBT)的训练与验证过程和上述操作类似，这里不再赘述。这四种分类器依次训练：各自正确率和AUC的结果如下，该数据也会保存在csv文件中：

```
LogisticRegression classifier Accuracy:0.7630
LogisticRegression classifier Auc:0.8366
DecisionTree classifier Accuracy:0.7571
DecisionTree classifier Auc:0.7431
Random Forest classifier Accuracy:0.7754
Random Forest classifier Auc:0.8406
GBT classifier Accuracy:0.7652
GBT classifier Auc:0.8424
```

4、可视化

Pyecharts是一款将python与echarts结合的数据可视化工具，本项目使用pyecharts来对第3步得到的分析结果进行可视化，代码均在echarts_show.py中，可在Pycharm中直接执行。

(1)播放量最多的数据可视化

首先分别从存放视频播放量次数和up总播放次数的两个csv文件中读取数据，为了最后布局的美观，对每一行字数较多的文本中使用add_line函数插入一些换行符：

```
def add_line(sentence,interval=6):
    sentence = sentence.replace(" ","")
    string = "(.{" + str(interval) + "})"
    sentence = re.sub(string,"\\1\n",sentence)
    return sentence
```

```
# 读取视频的数据
video_data = pd.read_csv(video_data_csv)
# 对长标题进行换行
video_data['title'] = [add_line(i) for i in video_data['title']]
# 读取up主的数据
up_data = pd.read_csv(up_data_csv)
# 对长的名字进行换行
up_data['up'] = [add_line(i) for i in up_data['up']]
```

接着对视频和up主数据分别绘制柱状图，其中Bar()表示用柱状图可视化数据，add_xaxis设置横坐标，add_yaxis设置纵坐标，set_global_opts设置例如标题、字体大小等的全局变量：

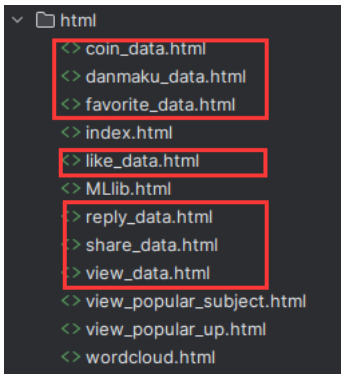
```
# 绘制柱状图
video_bar = (Bar(init_opts=opts.InitOpts(theme=ThemeType.ESSOS,width="1200px"),)
             .add_xaxis(video_data['title'].tolist())
             .add_yaxis('观看次数',video_data['view_data'].tolist())
             .set_global_opts(title_opts=opts.TitleOpts(title='观看次数最多的视频',pos_left='center',pos_top='5%'),
                              toolbox_opts=opts.TitleOpts(),
                              legend_opts=opts.LegendOpts(is_show=False),
                              xaxis_opts=opts.AxisOpts(axislabel_opts=opts.LabelOpts(rotate=0,font_size=10),))
)
```

```
# 绘制柱状图
up_bar = (Bar(init_opts=opts.InitOpts(theme=ThemeType.ESSOS,width="1200px"))
              .add_xaxis(up_data['up'].tolist())
              .add_yaxis('观看次数',up_data['view_data'].tolist())
              .set_global_opts(title_opts=opts.TitleOpts(title='观看次数最多的up主',pos_left='center',pos_top='5%'),
                               toolbox_opts=opts.TitleOpts(),
                               legend_opts=opts.LegendOpts(is_show=False),
                               xaxis_opts=opts.AxisOpts(axislabel_opts=opts.LabelOpts(rotate=0,font_size = 10),))
)
```

最后通过Page()类将两个柱状图按照上下排列的顺序布局在同一个页面中，渲染成html文件：

```
# 通过Page将两个柱状图按上下分布保存在同一个html中
save_dir = os.path.join(base_dir,'view_data.html')
page = Page(layout=Page.SimplePageLayout)
page.add(video_bar,up_bar)
page.render(save_dir)
```

有关投币数、弹幕数、收藏数、点赞数、回复数的数据可视化流程如上，最终可视化结果会保存在各自的html文件中：



(2)收录次数可视化

对收录进《每周必看》栏目次数最多的视频分区主题分布进行可视化，首先从csv文件中读入数据，同样对长标题添加换行符：

```
# 读入数据
video_data_csv = os.path.join(data_dir,'top_popular_subject.csv')
video_data = pd.read_csv(video_data_csv)
save_dir = os.path.join(base_dir,'view_popular_subject.html')
# 对长标题添加换行符
video_data_short = [add_line(i,2) for i in video_data['tname']]
```

按照每种分区的收录次数，绘制柱状图：

```
# 绘制柱状图
video_popular_bar = (Bar(init_opts=opts.InitOpts(theme=ThemeType.ESSOS,width='600px'),)
              .add_xaxis(video_data_short)
              .add_yaxis('上榜次数',video_data['popular_subject_times'].tolist())
              .set_global_opts(title_opts=opts.TitleOpts(title='上榜次数前十的话题',pos_right='18%',pos_top='5%'),
                               toolbox_opts=opts.TitleOpts(),
                               legend_opts=opts.LegendOpts(is_show=False),
                               xaxis_opts=opts.AxisOpts(axislabel_opts=opts.LabelOpts(rotate=0,font_size = 10),))
)
```

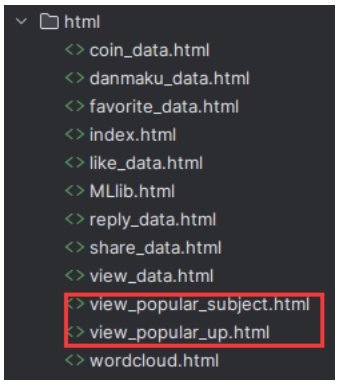
根据分区收录的次数，绘制富文本饼图：

```
# 绘制富文本饼图
video_popular_pie = (
    Pie(init_opts=opts.InitOpts(theme=ThemeType.ESSOS))
        .add(
        "",
        [list(z) for z in zip(video_data['tname'].tolist(), video_data['popular_subject_times'].tolist())],
        radius=["30%", "55%"],
        center=["25%", "50%"],
        label_opts=opts.LabelOpts(
            position="outside",
            formatter="{a|{a}}{abg|}\n{hr|}\n {b|{b}: }{c}  {per|{d}%}  ",
            background_color="#eee",
            border_color="#aaa",
            border_width=1,
            border_radius=3,
            rich={
                "a": {"color": "#999", "lineHeight": 22, "align": "top"},
                "abg": {
                    "backgroundColor": "#e3e3e3",
                    "width": "50%",
                    "align": "right",
                    "height": 22,
                    "borderRadius": [4, 4, 0, 0],
                },
                "hr": {
                    "borderColor": "#aaa",
                    "width": "50%",
                    "borderWidth": 0.5,
                    "height": 0,
                },
                "b": {"fontSize": 16, "lineHeight": 33},
                "per": {
                    "color": "#eee",
                    "backgroundColor": "#334455",
                    "padding": [2, 3],
                    "borderRadius": 2,
                },
            },
        ),
    )
        .set_global_opts(title_opts=opts.TitleOpts(title="上榜次数前十的话题占比",pos_left='20%',pos_top='5%'),\
        legend_opts=opts.LegendOpts(is_show=False,type_="scroll",orient='vertical',pos_top='10%',pos_right='50%'))
)
```

最后通过Grid()类将两个柱状图按照左右排列的顺序布局在同一个页面中，渲染成html文件：

```
# 通过Grid将两个结果按照左右位置进行布局，渲染到html文件中
grid = Grid(init_opts=opts.InitOpts(theme=ThemeType.ESSOS,width='1400'))
grid.add(video_popular_bar,grid_opts=opts.GridOpts(pos_left="60%"))
grid.add(video_popular_pie,grid_opts=opts.GridOpts(pos_left="50%"))
grid.render(save_dir)
```

有关收录次数最多的up主数据可视化流程同上，各自的结果都保留在html目录：



(3)词频信息可视化
从csv文件中读出标题的词频信息，并将结果转为list，方便后续的处理：

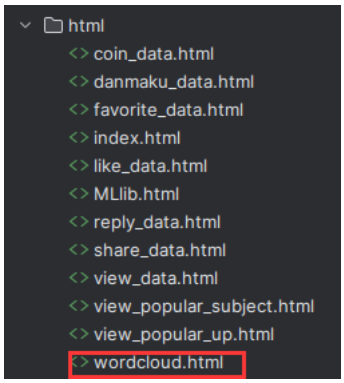
```
# 读取数据
title_words = pd.read_csv(os.path.join(base_dir,'title_word.csv'))
# 把数据转为list
title_words_freq = map_to_list(title_words)
```

使用pyecharts提供的WordCloud类绘制词云图：

```
# 绘制词云
title_wc = (
WordCloud(init_opts=opts.InitOpts(theme=ThemeType.ESSOS))
.add(
    series_name="",
    shape='circle',
    data_pair=title_words_freq,            #数据来源
    word_size_range=[16, 66],    #字号大小范围设置
    textstyle_opts=opts.TextStyleOpts(
    color='#dcdada'
    ),
)
.set_global_opts(     #全局设置项
    title_opts=opts.TitleOpts(
        title="标题常见词", title_textstyle_opts=opts.TextStyleOpts(font_size=23),pos_right='center'
    ),
    tooltip_opts=opts.TooltipOpts(is_show=False),
)
.set_series_opts(
    areastyle_opts=opts.AreaStyleOpts(opacity = 0)
)
)
```

视频简介与推荐理由的操作类似，最后得到的三个词云再通过Page()类，按照上中下的顺序布局在同一个页面中，渲染成html文件：

```
# 通过Page()，把三个可视化结果按照上中下顺序布局，渲染成html文件
page = Page(layout=Page.SimplePageLayout)
page.add(title_wc,desc_wc,rcmd_reason_wc)
page.render(save_dir)
```



(4) 机器学习库分析结果可视化

变量之间的相关性可通过热力图来呈现，首先从csv文件中读取相关矩阵：

```
# 读取相关矩阵
cor_mat_csv = pd.read_csv(os.path.join(base_dir, 'cor_matrix.csv'))
# 保留4位小数
cor_mat_csv = cor_mat_csv.round(4)
cor_mat_csv = cor_mat_csv.values
```

把数据处理为列表格式，列表中每一个元素[i,j,value]表示变量i与变量j之间的相关性为value：

```
# 对数据数据为[i,j,value]的格式，表示变量i和变量j之间的相关性为value
variable_list = ['view', 'danmaku', 'reply', 'favorite', 'coin', 'share', 'like']
data = [[i, j, cor_mat_csv[i][j]] for i in range(len(variable_list)) for j in range(len(variable_list))]
```

绘制热力图：

```
# 绘制热力图
heatmap = (
    HeatMap(init_opts=opts.InitOpts(theme=ThemeType.ESSOS, width="1200px"))
    .add_xaxis(variable_list)
    .add_yaxis("", variable_list, data)
    .set_global_opts(
        title_opts=opts.TitleOpts(title="自变量相关性", pos_right='center', pos_top='2%'),
        visualmap_opts=opts.VisualMapOpts(max_=1, pos_right='5%', pos_bottom='20%'),
        xaxis_opts=opts.AxisOpts(name="Features", axislabel_opts=opts.LabelOpts(font_size=10), ),
        yaxis_opts=opts.AxisOpts(name="Features", axislabel_opts=opts.LabelOpts(font_size=10)),
    )
)
```

而分类器的性能对比可通过双柱状图来表示，同样，首先从csv文件中读取各个分类器的性能，并对分类器的名称与Acc、Auc进行操作，确保可视化结果的美观：

```
# 读取分类器的性能指标
comparison_csv = pd.read_csv(os.path.join(base_dir, 'comparison.csv'))
# 保留4位有效数据
comparison_csv = comparison_csv.round(4)
# 对名字较长的分类器添加换行符
comparison_csv['classifier'] = [add_line_eng(i) for i in comparison_csv['classifier']]
```

绘制双柱状图，通过两次add_yaxis将Acc与Auc的数据添加到图上：

```
# 绘制双柱状图
comparison_bar = (Bar(init_opts=opts.InitOpts(theme=ThemeType.ESSOS, width="1200px"))
                  .add_xaxis(comparison_csv['classifier'].tolist())
                  .add_yaxis('Acc', comparison_csv['Acc'].tolist())
                  .add_yaxis('Auc', comparison_csv['Auc'].tolist())
                  .set_global_opts(
    title_opts=opts.TitleOpts(title='不同分类器的性能比较', pos_left='center', pos_top='1%'),
    toolbox_opts=opts.TitleOpts(),
    legend_opts=opts.LegendOpts(pos_left='10%', pos_top='6%'),
    xaxis_opts=opts.AxisOpts(axislabel_opts=opts.LabelOpts(rotate=0, font_size=10), ),
    )
                  )
```

最后再通过Page()类将两个结果按照上下的顺序布局，渲染成html文件：

```
# 通过Page()类，将热力图和双柱状图按照上下的排列顺序渲染成html文件
page = Page(layout=Page.SimplePageLayout)
page.add(heatmap, comparison_bar)
page.render(save_dir)
```



main函数依次执行上述操作，所有html文件均保存在/html目录中，代码如下：

```
if __name__ == '__main__':
    # 分析结果保存路径
    rendered_data = './static'
    # 项目保存路径
    render_html = './html'
    os.makedirs(render_html,exist_ok=True)
    # 数据可视化
    show_statistic(rendered_data,render_html)
    # 上榜视频与up主信息可视化
    show_popular(rendered_data,render_html)
    # 词频可视化
    show_word_clowd(rendered_data,render_html)
    # 机器学习组件可视化
    show_Mllib(rendered_data,render_html)
```

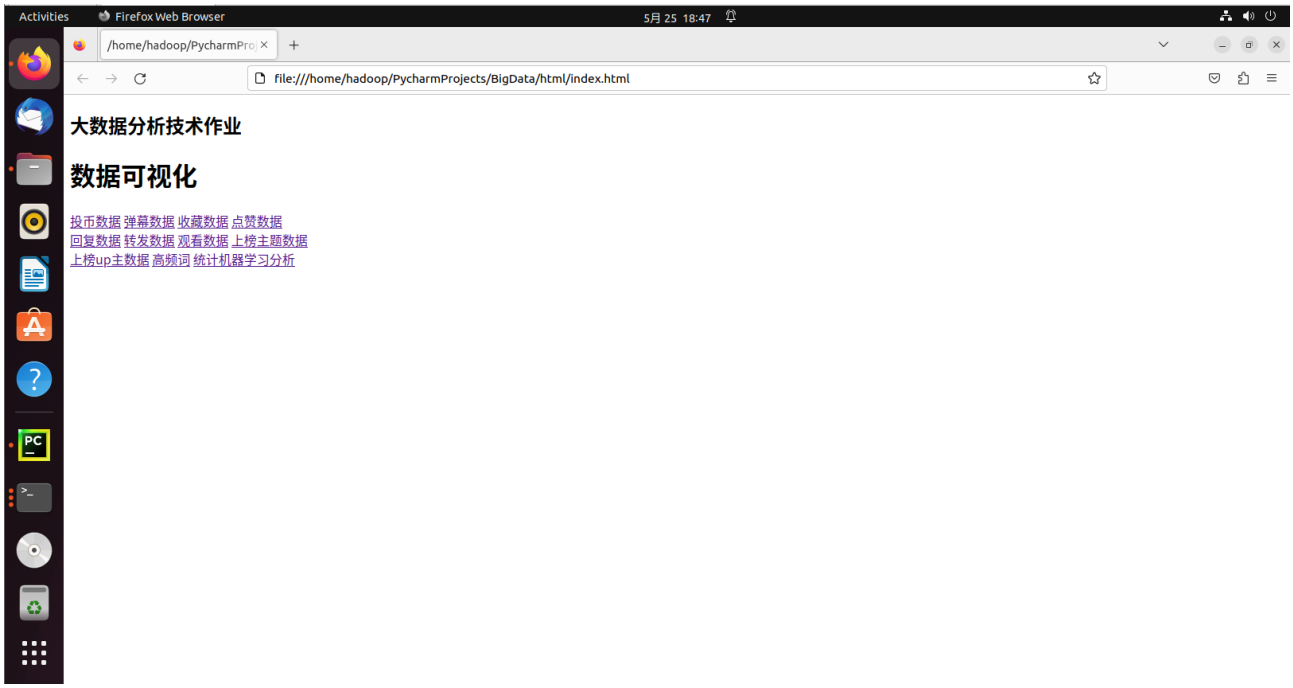
最后，为了方便可视化结果的查看，在/html目录下编写index.html文

件，通过按键超链接，设置不同可视化结果的跳转，index.html的主要代码如下：

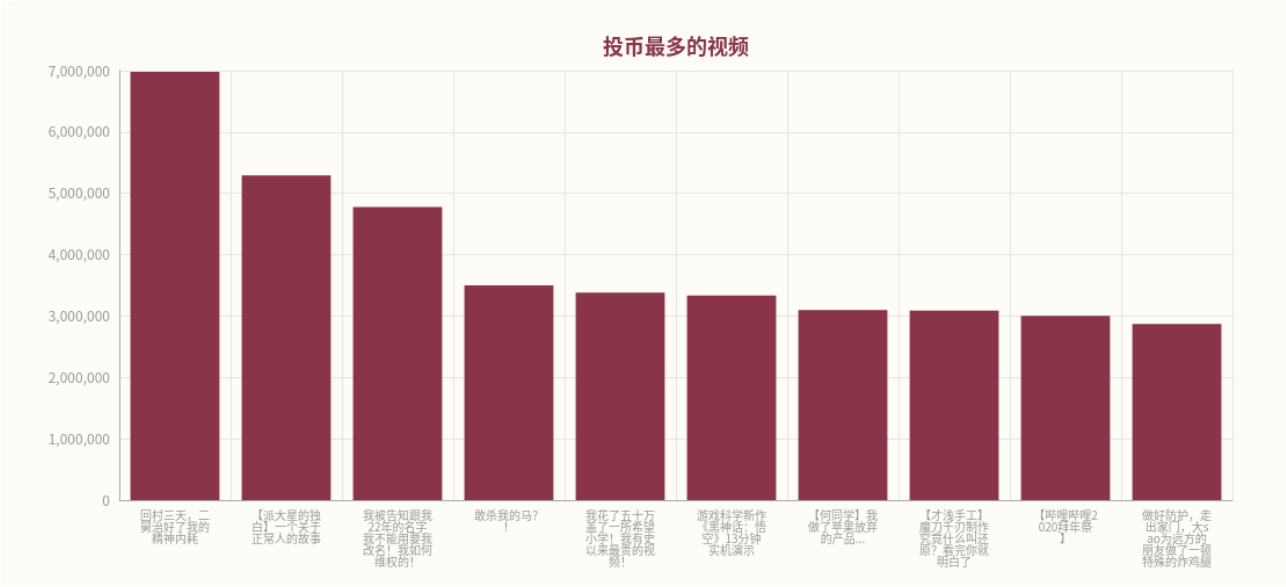
```
<div class = "header-info">
    <h1>数据可视化</h1>
    <div class = "buttons">
        <a class="button" href="coin_data.html" target="_blank">投币数据</a>
        <a class="button" href="danmaku_data.html" target="_blank">弹幕数据</a>
        <a class="button" href="favorite_data.html" target="_blank">收藏数据</a>
        <a class="button" href="like_data.html" target="_blank">点赞数据</a>
    </div>
    <div class = "buttons">
        <a class="button" href="reply_data.html" target="_blank">回复数据</a>
        <a class="button" href="share_data.html" target="_blank">转发数据</a>
        <a class="button" href="view_data.html" target="_blank">观看数据</a>
        <a class="button" href="view_popular_subject.html" target="_blank">上榜主题数据</a>
    </div>
    <div class = "buttons">
        <a class="button" href="view_popular_up.html" target="_blank">上榜up主数据</a>
        <a class="button" href="wordcloud.html" target="_blank">高频词</a>
        <a class="button" href="MLlib.html" target="_blank">统计机器学习分析</a>
    </div>
</div>
```

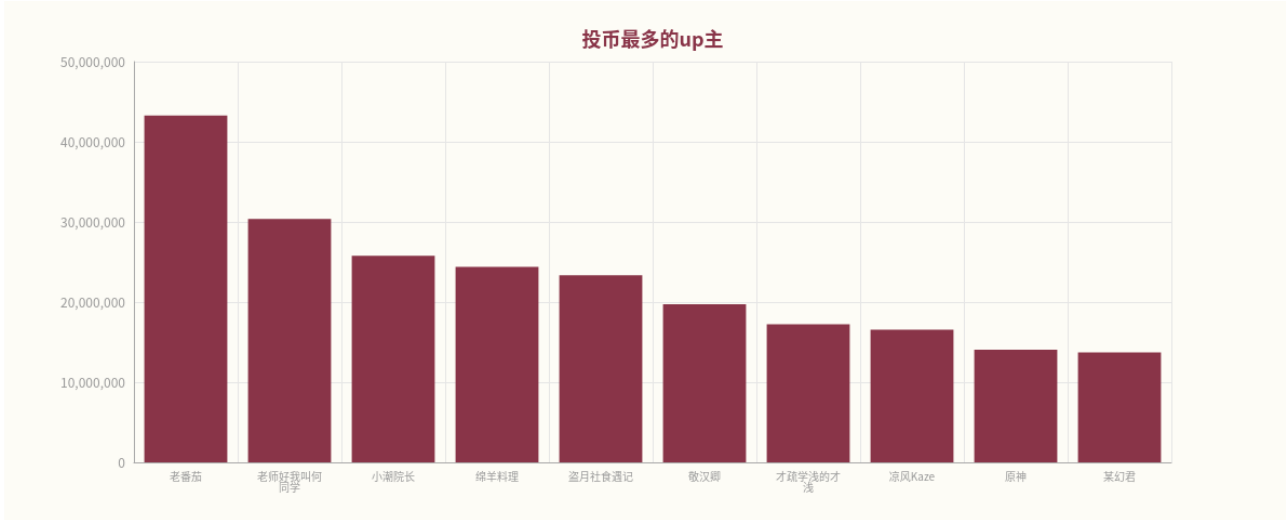
四、可视化结果

1、打开工程目录下/html/index.html的页面如下，点击文字，既可实现各个页面的跳转:



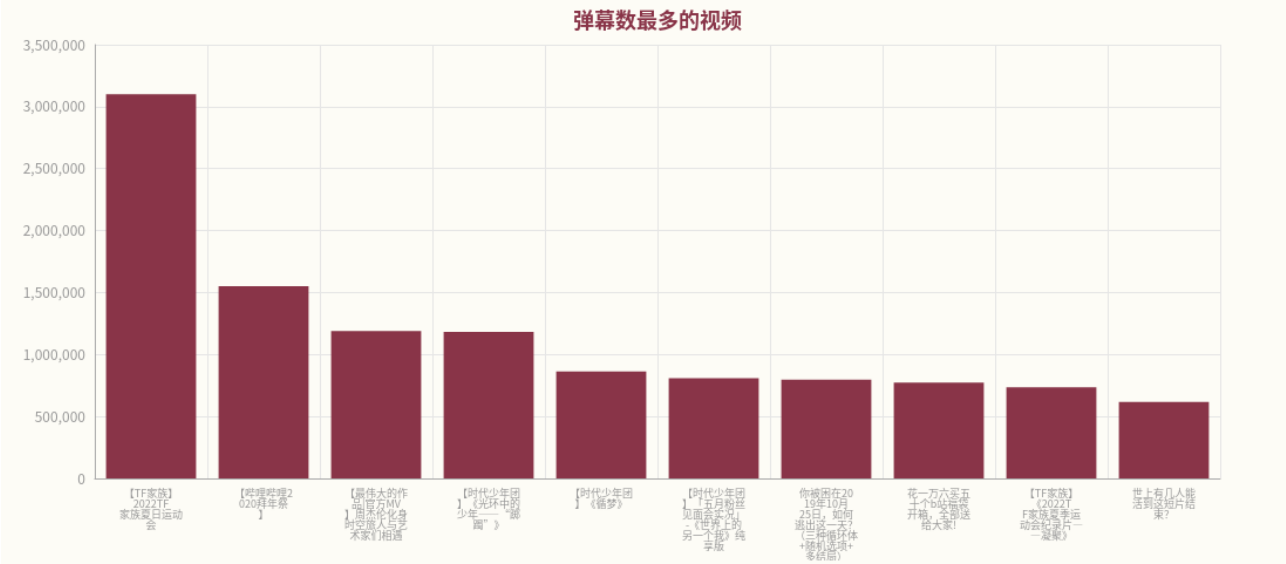
2、投币数据可视化

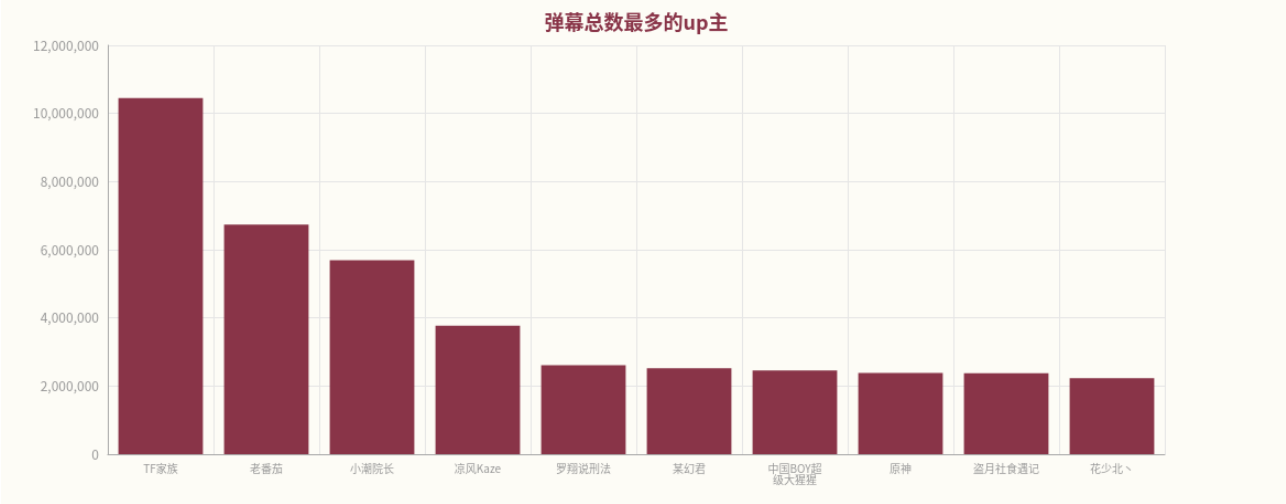




视频《回村三天，二舅治好了我的精神内耗》得到的硬币数量最多，up主老番茄得到的总投币量最多。

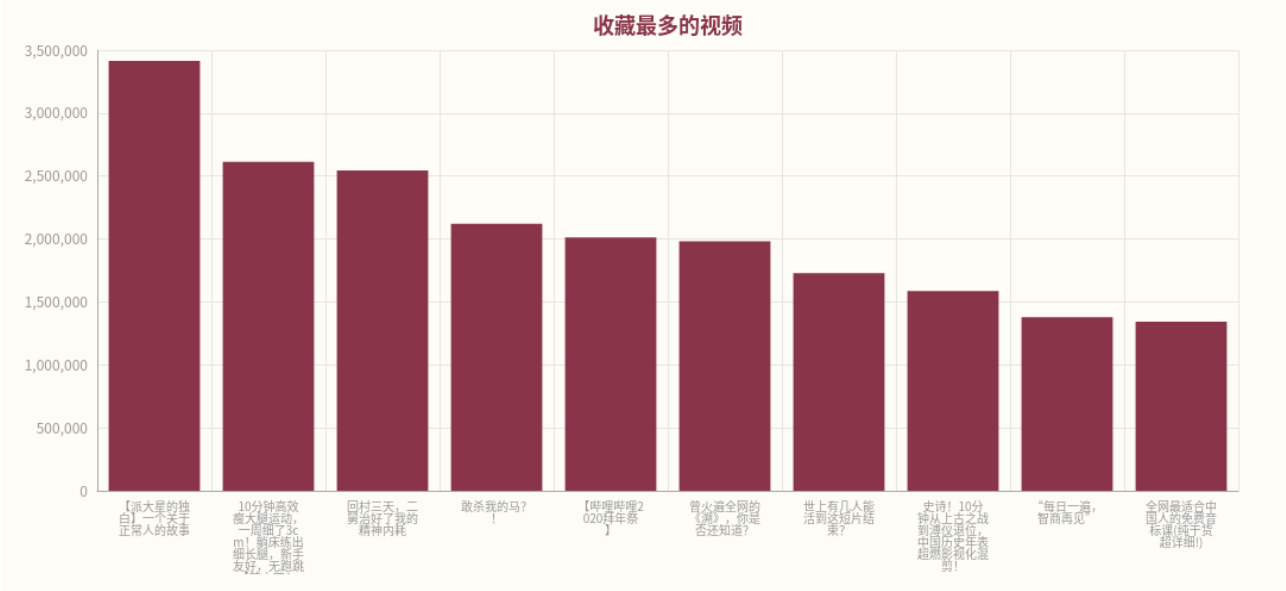
3、弹幕数据可视化

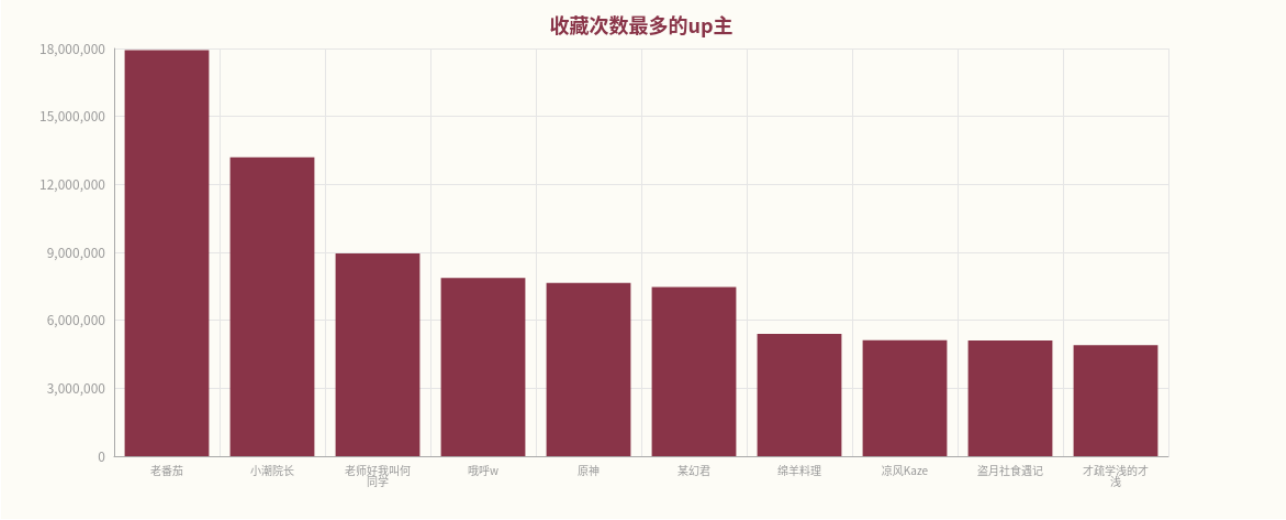




在弹幕数据中，《【TF家族】2022TF家族夏日运动会》以及up主TF家族发布的视频有最多的弹幕。

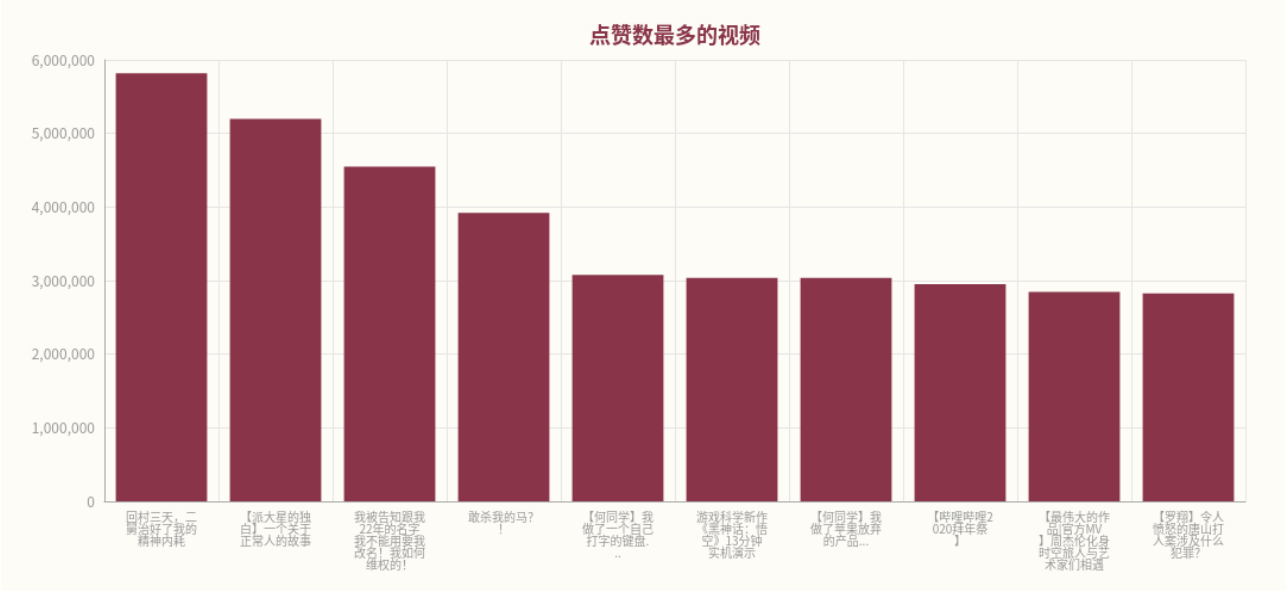
4、收藏数据可视化

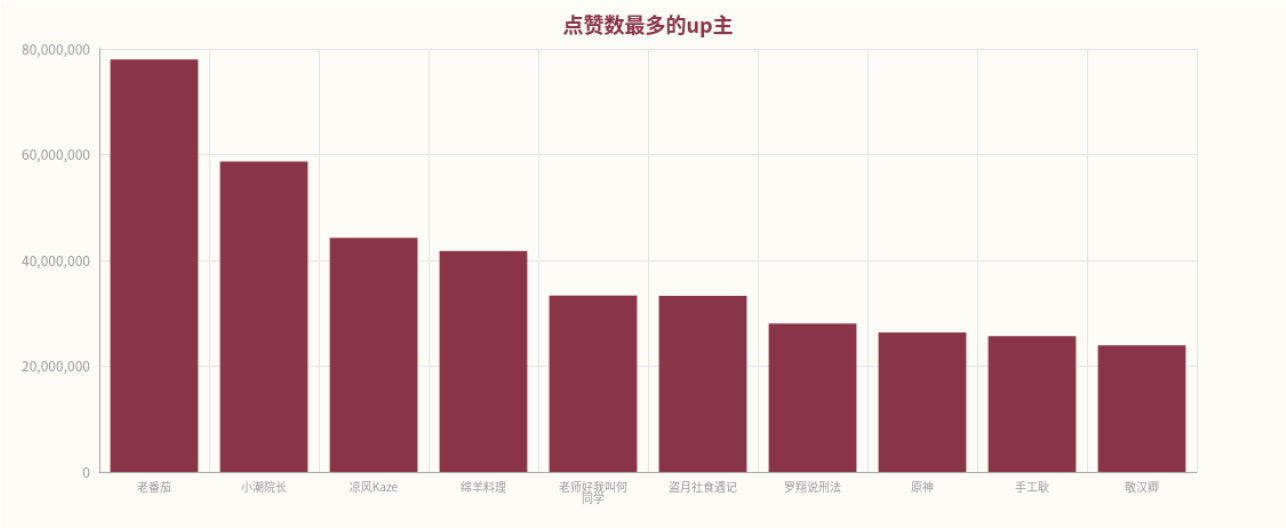




有关视频的收藏数据，《【派大星的独白】一个关于正常人的故事》和up主老番茄有着最多的收藏数。

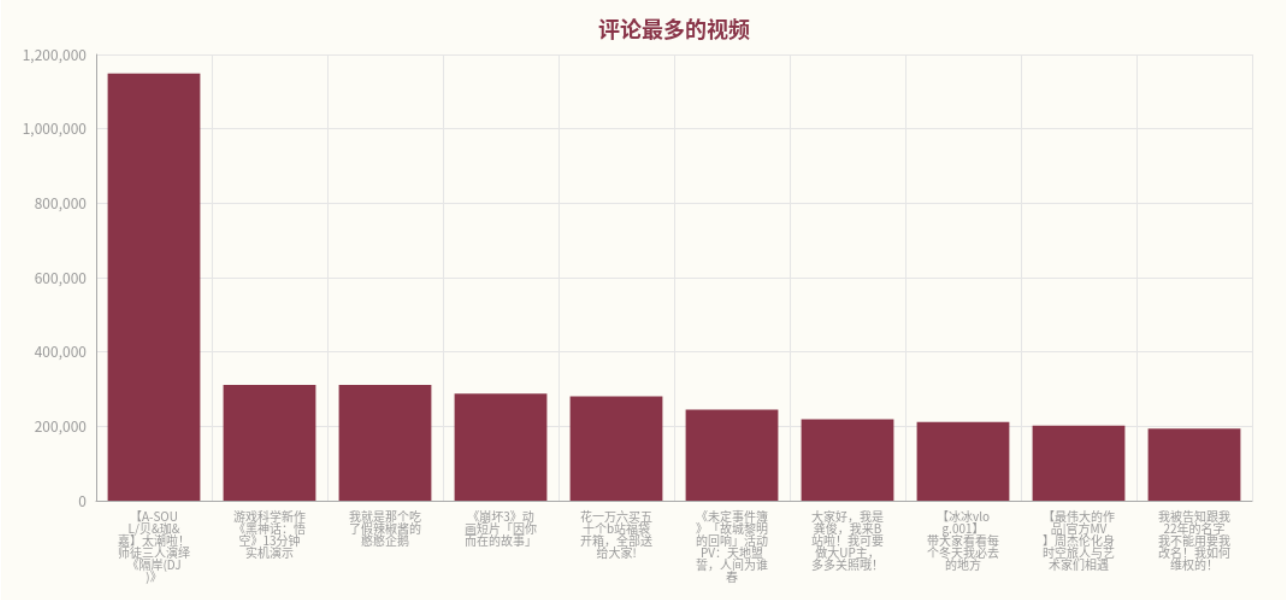
5、点赞数据可视化

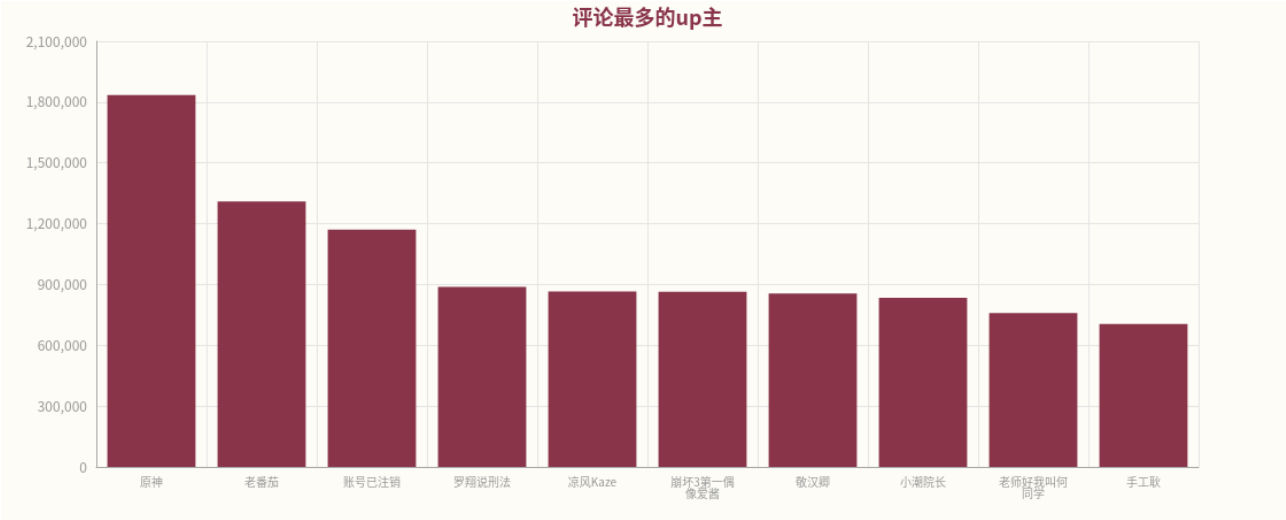




在点赞数据中，视频《回村三天，二舅治好了我的精神内耗》得到的最多点赞，在各up主中，老番茄的视频得到最多点赞。

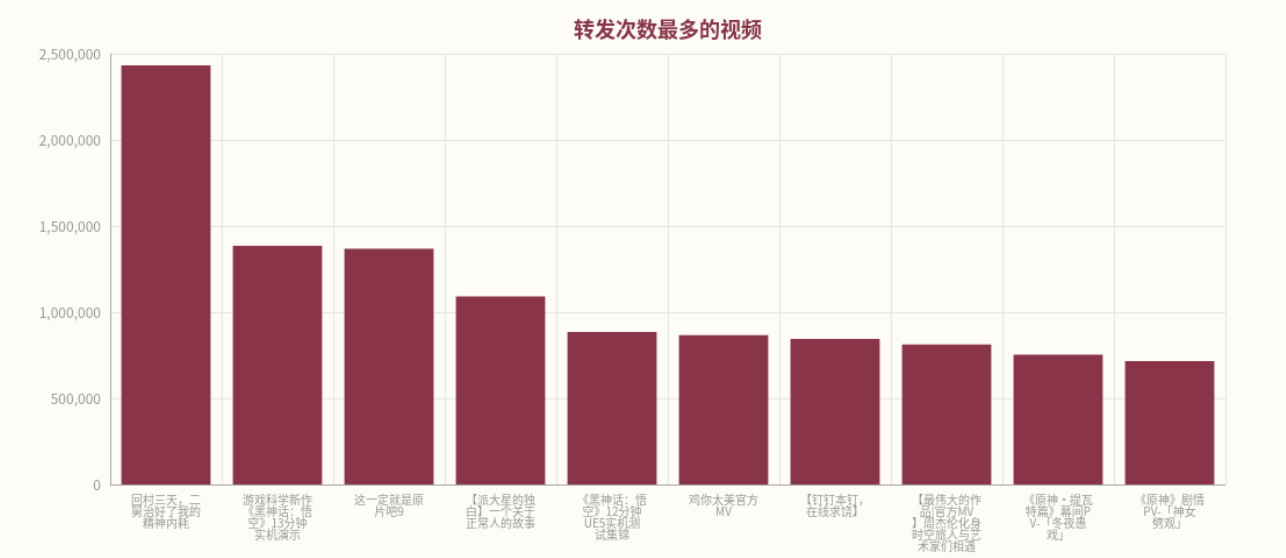
6、评论数据可视化

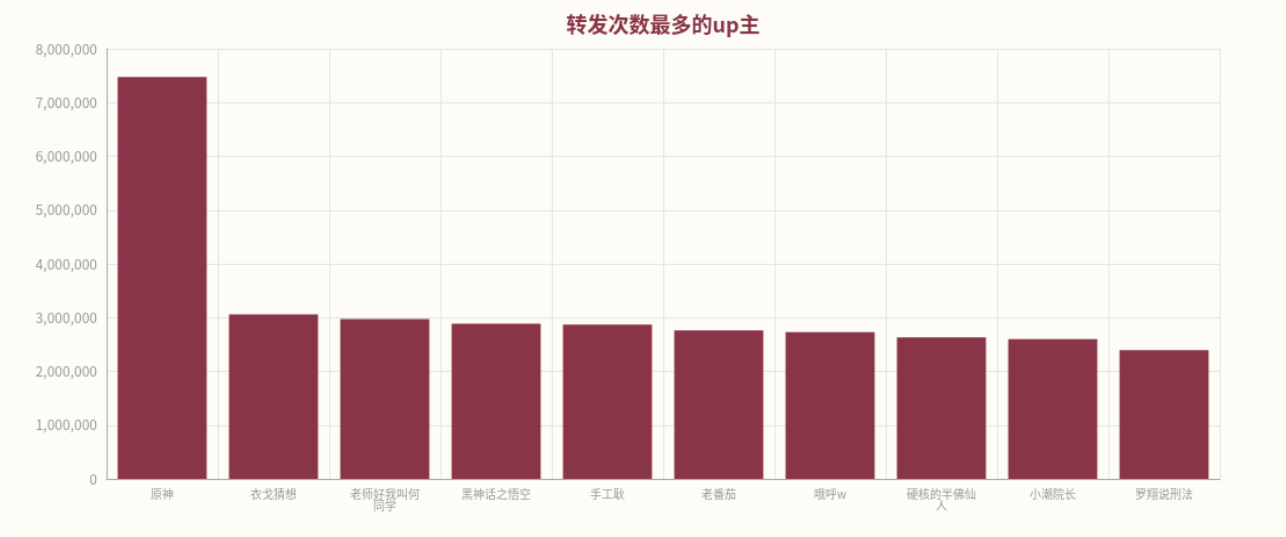




【A-SOUL/贝&珈&嘉】太潮啦！师徒三人演绎《隔岸 (DJ)》视频得到最多的评论，而账号原神发布的视频得到评论最多。

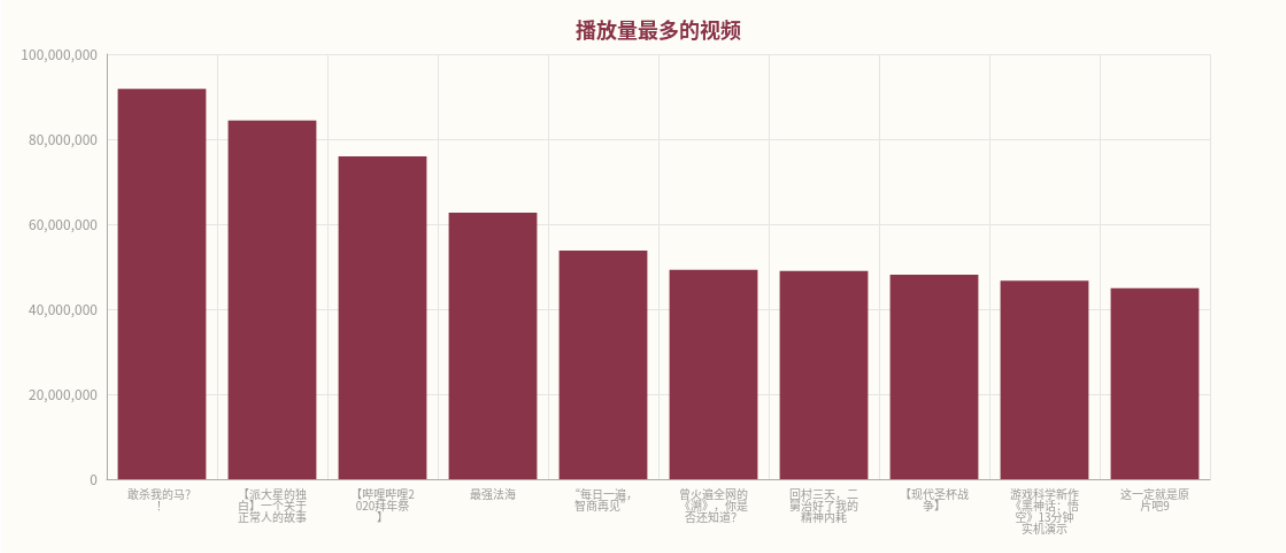
7、转发次数可视化

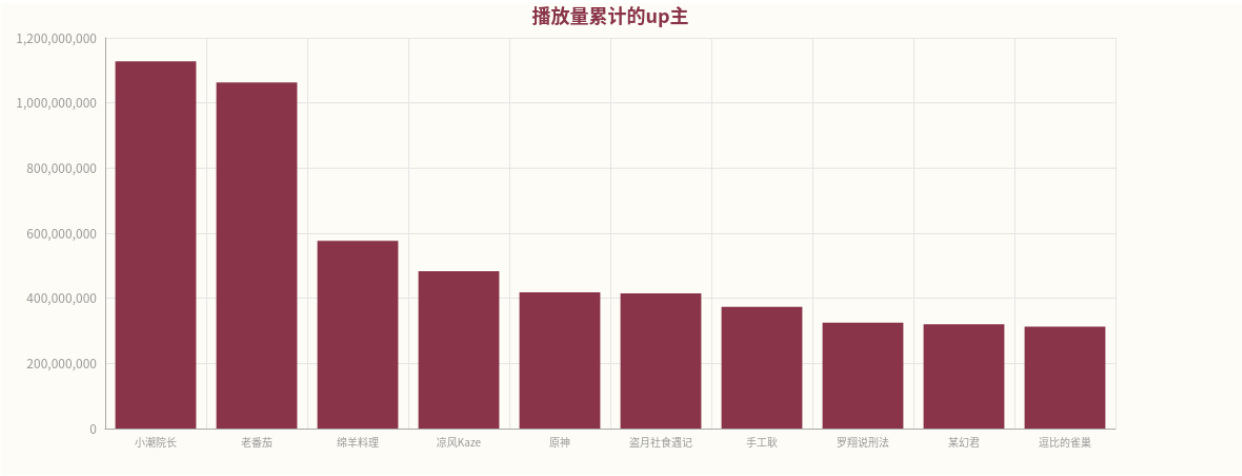




在转发的数据中，视频《回村三天，二舅治好了我的精神内耗》的分享次数最多，原神发布的视频分享总数最多。

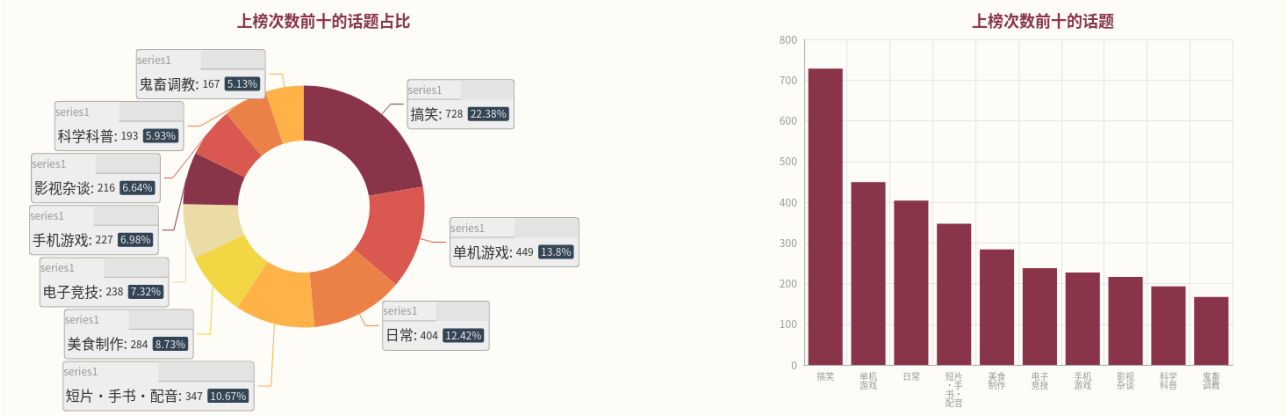
8、播放量数据可视化



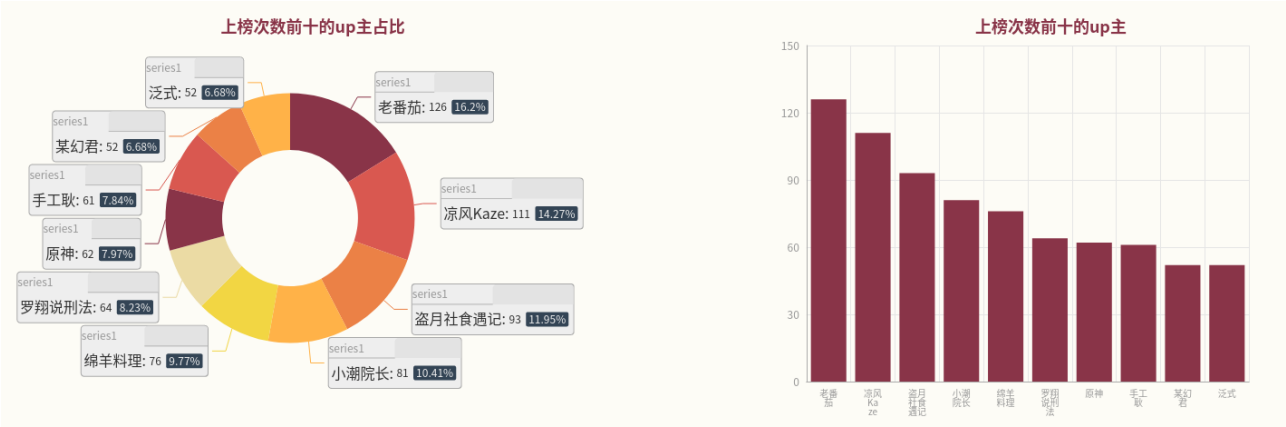


在视频的播放量方面，视频《敢杀我的马？》播放量最多，up主小潮院长的播放量累计最大。

9、视频分区占比可视化



10、up占比可视化


上榜次数前十的up主占比
series1
泛式: 52 6.68%
series1
老番茄: 126 16.2%
series1
某幻君: 52 6.68%
series1
手工耿: 61 7.84%
series1
凉风Kaze: 111 14.27%
series1
原神: 62 7.97%
series1
罗翔说刑法: 64 8.23%
series1
盗月社食遇记: 93 11.95%
series1
绵羊料理: 76 9.77%
series1
小潮院长: 81 10.41%
上榜次数前十的up主
150
120
90
60
30
0
老番茄
凉风Kaze
盗月社食遇记
小潮院长
绵羊料理
罗翔说刑法
原神
手工耿
某幻君
泛式

11、词频统计结果

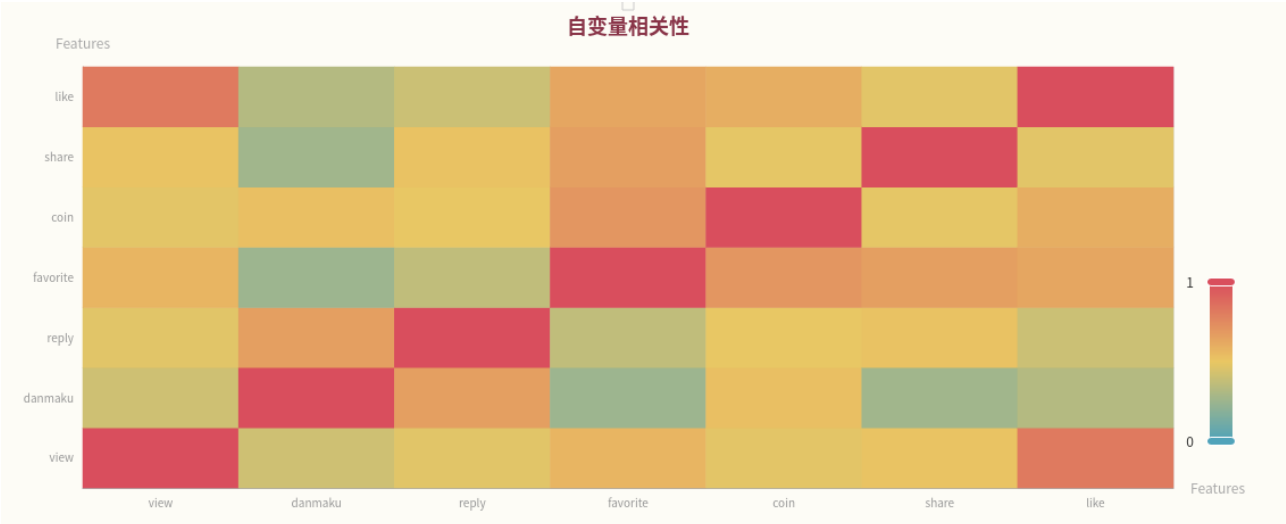

标题常见词


视频简介常见词

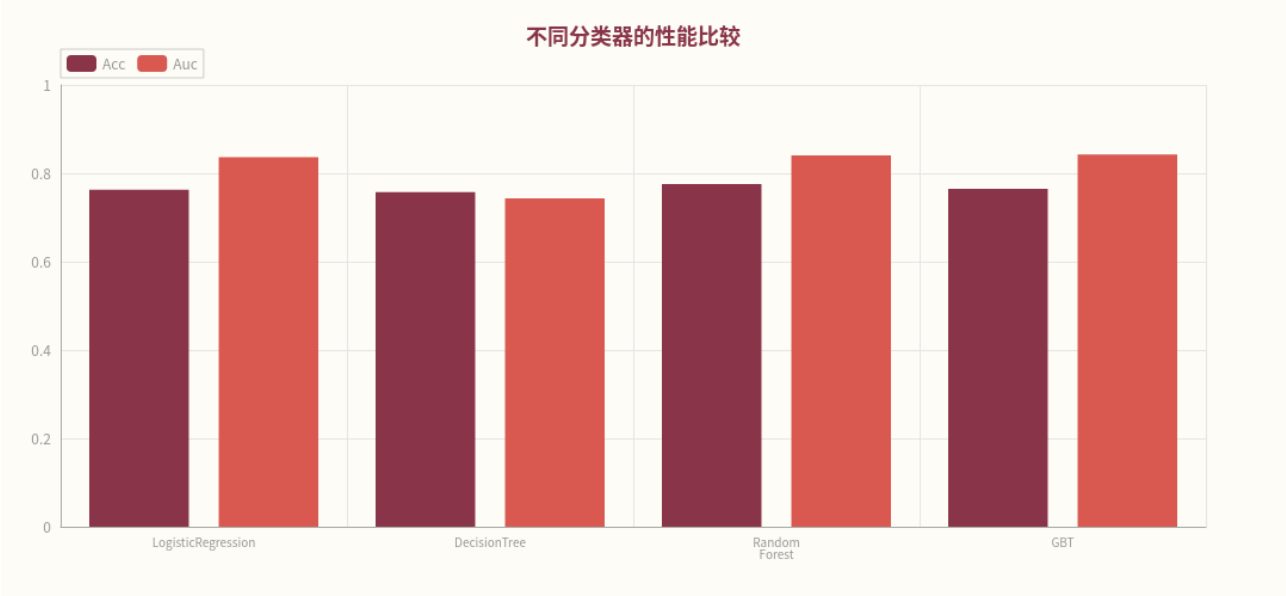


从上面三个词云，可以看到b站的语言文字以口语化、诙谐化为主，并且游戏领域例如原神等更受欢迎。

12、机器学习可视化结果



从热力图来看，几个变量之间的关系基本符合预期：观看次数越多的视频，得到的收藏和点赞可能越多，它们之间的相关性越大；而弹幕数和评论数也有较大的相关性，因为这两个数据都是反应视频与观众的互动性。



从几个分类器的性能来看，仅通过视频的播放量、评论数、弹幕数等可以初步对它们的热搜排名进行一个分类，但是这种分类准确率还有进一步的提升空间，可能还需要再考虑到发布视频up的粉丝数量、题材等等的信息。

五、遇到的问题与解决方法

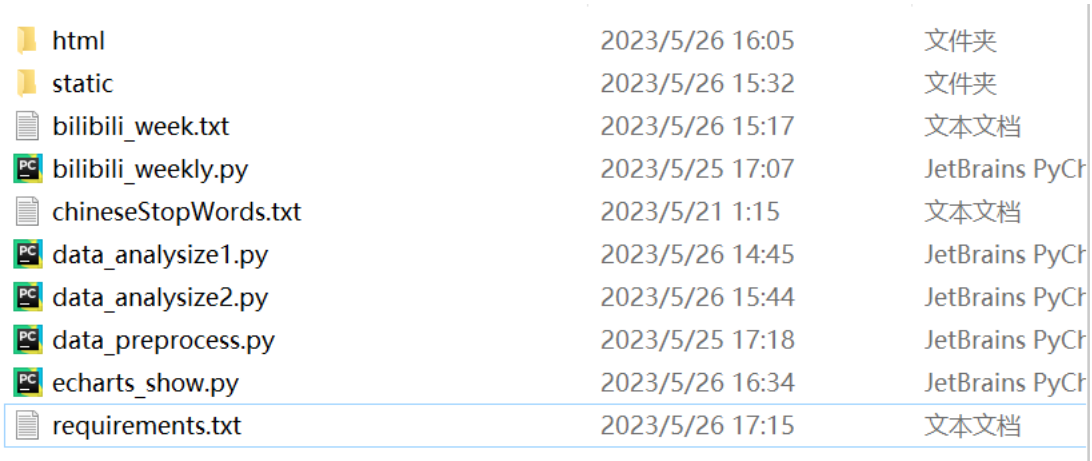
1、关于spark的版本，项目是在conda里新建了一个虚拟环境，虚拟环境里面需要安装pyspark，安装成功后实际使用的spark版本和pyspark的版本一致，而与虚拟机上安装的spark无关。因此使用python+虚拟环境完成作业时，可以直接在虚拟环境里pip install pyspark==3.2.0解决spark的安装，这样更加简便快捷。
2、关于爬虫的代码，在windows系统下总是会出现奇奇怪怪的问题，比如“未能连接”或者“拒绝请求”，需要反复执行代码才有几率成功执行，但是反而在Linux环境下爬虫不会出现这些错误，因此最后数据的爬取也是在虚拟机中进行。
3、b站每个视频的数据，比如播放量、评论数、弹幕数等等都是会不断更新的，因此每一次运行代码得到的结果例如播放量、转发次数等都会不同，报告及可视化中的结果显示的是截止至5月26日各个视频的数据情况。但就整体而言，短时间内微小的变化量不影响整体的分析结果。

六、实验总结

这次实验我粗略体验了大数据分析的流程，就本次使用的数据集和分析结果来看， b站作为一个以娱乐为主的视频平台，搞笑、明星、游戏等相关视频会更受用户的喜爱，但是也不难看出，一些日常生活类的视频例如《二舅》等，通过讲述日常或身边的人物故事，思考生活的意义与本质，容易与年轻人产生共鸣，同样也受到较高的关注度。整个分析思考过程中，我学会了如何爬虫、如何根据自己的目标选择合适的数据并对其进行清洗，如何使用hadoop和spark工具对数据进行分析以及最后如何把分析得到的结果进行可视化，收获颇丰。

补充说明

1、代码文件目录为BigData，数据集也包含在其中。文件目录格式如下：



data：存放爬虫得到json文件的路径，因该数据过大，提交时代码时把它删除；
html：存放可视化结果的路径，其中index.html集合了其他各个网页的跳转链接；
static：使用spark得到的分析结果；
bilibili_week.txt：对爬虫数据清洗后的数据集，也是最终上传到HDFS的数据集；

bilibili_weekly.py：爬虫代码；
chineseStopWords.txt：中文停用词，用于对文本进行分词；
data_analysize1.py：使用spark sql组件分析的代码；
data_analysize2.py：使用spark MLlib组件分析的代码；
data_process.py：数据处理代码；
echarts_show.py：可视化代码；
requirements.txt：代码所需的第三方库。
2、参考资料
(1)b站爬虫代码参考：
https://www.heywhale.com/mw/project/6059c0f0c910a9001581c98b
(2)python spark数据分析案例：https://dblab.xmu.edu.cn/blog/2738/
(3)pyecharts可视化文档：https://pyecharts.org/#/zh-cn/intro